Phase 12A - INVESTIGATE FAILED DOCUMENT QA QUERY

A. ROOT CAUSE

The "Tower 7 Level 02 Floor GA" query failed due to a two-fold architectural flaw in the structured extraction integration:

1. **Extraction Integration Gap:** The previous AI successfully tested the `extract_and_store` PDF hierarchical table parser in a standalone script (`structured_test.py`), but completely forgot to integrate it into the real `process_document` ingestion pipeline. The live system was still using the generic markdown table extractor which populates `schedule_rows` rather than the deterministic `structured_records` table required by the Query Router (`intelligence/structured_query.py`).
2. **Brittle PDF Parsing:** Even if integrated, the standalone script hardcoded `row[1]` for `TOWER` and `row[3]` for `LEVEL`. Due to PyMuPDF cell bounding algorithms, "Tower 6" happened to align with `row[3]` for the LEVEL field, but "Tower 7" aligned with `row[4]`. The script silently skipped all rows for Tower 7 and Tower 8, producing 0 records.

B. EVIDENCE

- Tracing `main.py -> extract_structured_tables_from_pdf` revealed no insertion into `StructuredRecord`.
- Running `SELECT DISTINCT tower FROM structured_records` after a manual script run showed only Towers 1 through 6.
- Dumping the exact row containing "TOWER 7" directly from PyMuPDF returned: `['', 'TOWER 7', None, '', 'LEVEL 01']` -> Tower is at index 1, Level is at index 4 (not index 3).

C. FILES INVOLVED

- `knowledge/extraction.py` (Missing `run_structured_record_extraction` injection and hardcoded indices).
- `intelligence/structured_query.py` (Correctly parsed query but queried an unpopulated table).

D. FIX

1. Embedded the `extract_and_store` hierarchy-reconstruction logic directly into `knowledge/extraction.py` as `run_structured_record_extraction()`.
2. Modified `index_structured_tables` to query the physical `pdf_path` from the SQLite `documents` table and run the dedicated PyMuPDF extractor on every upload to populate `StructuredRecord`.
3. Replaced the brittle `row[1]` and `row[3]` hardcoding with a dynamic scan of each row array to detect `'TOWER'` and `'LEVEL'`.
4. Dynamically calculated the data start index by checking columns for unit signatures

(`sqm`, `unit`, `%`) rather than assuming data starts at `row[4]`.

E. REGRESSION TESTS

- **Tower 6 Level 02 GFA**: PASS (2407.0 sqm)
- **Tower 7 Level 02 Floor GA**: PASS (2111.0 sqm)
- **Tower 7 Level 02 GFA**: PASS (2053.0 sqm)
- **Tower 7 Level 02 NISA**: PASS (1665.0 sqm)
- **Missing tower / Missing level**: PASS (`AMBIGUOUS` returned successfully by SQLite layer fallback logic).
- Tested directly via deterministic query function to guarantee zero Qwen generation or hallucination.

F. RE-UPLOAD TEST

Re-uploading the exact same document now seamlessly drops old `StructuredRecord` rows by `project_id` and `filename` before inserting the newly parsed ones. Duplicates are prevented natively.

G. FINAL STATUS

DOCUMENT QA QUERY: "Tower 7 Level 02 Floor GA" **Expected:** Correct deterministic value + unit + provenance. **Actual:** Returns deterministic SQLite HIT (Value: 2,111.0 sqm, Page 6) without invoking RAG fallback.

Status: RESOLVED.